

Reviewer Pack — Minimal Rank of Tropicalized Quantum Groups Bounds Circuit D...

Entry 8beca60a4d2f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 23:39:44 UTC

Minimal Rank of Tropicalized Quantum Groups Bounds Circuit Depth for Ternary Operations

Entry ID: 8beca60a4d2f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 23:39:44 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Quantum Groups) **Field B** (complexity object): Complexity Theory: Boolean Circuit Complexity with Ternary Operations

Statement:

[‘For every n -ary boolean function f with monotone circuits of depth d using only addition, multiplication, and modulo 2 operations, the minimal rank $\rho(f)$ of its tropicalization is at least $\Omega(d \log n)$.’, ‘There exists a constant $c > 0$ such that for all n -ary boolean functions f , if their tropicalization has a rank less than $c d \log n$, then f can be computed by a monotone circuit of depth $< d$.’, ‘For any given n and d , there exists an n -ary boolean function f that is computable by a monotone circuit of depth d , but its tropicalization has a rank at least $c d \log n$.’]

Rationale (proposer’s reasoning):

[‘Tropicalizations of functions provide a bridge between the Boolean and tropical semirings. Quantum groups have been studied in algebraic structures that exhibit noncommutative properties relevant to quantum computation.’, ‘The conjecture bridges the gap by introducing the concept of minimal rank for tropicalized quantum groups, which could potentially lead to new insights in boolean circuit complexity with ternary operations.’, ‘Quantum groups may offer a novel approach to analyze circuits and provide a deeper understanding of their complexity.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 92eca5cd81e0e61f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for at least 95% of randomly generated n -ary boolean functions f with depth d , the minimal rank $\rho(f)$ of their tropicalization meets or exceeds the lower bound $\Omega(d \log n)$, and falsified if less than 5% of such functions meet this criterion.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry quantum groups AND boolean circuit complexity ternary operations - minimal rank tropicalization quantum groups AND depth bounds boolean circuits - quantum group tropicalization lower bound monotone circuits ternary operations

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1207.1925v1] Introduction to tropical algebraic geometry - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/1911.04516v1] Boolean lattices in finite alternating and symmetric groups - [http://arxiv.org/abs/2203.16571v3] Random quantum circuits are approximate unitary t -designs in depth $O(nt^{5+o(1)})$ - [http://arxiv.org/abs/2406.18700v4] Structure of sparse Boolean functions over Abelian groups, and its application to testing - [http://arxiv.org/abs/1406.3065v2] Lower Bounds for Tropical Circuits and Dynamic Programs - [http://arxiv.org/abs/1007.1875v2] Lower Bounds for Quantum Oblivious Transfer

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.7s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def tropicalize(f):
        n = int(math.log2(len(f)))
        T = [[max(f[i], f[j]) for j in range(2**n)] for i in range(2**n)]
        return T

    def rank(T):
        m, n = len(T), len(T[0])
        if m != n:
            raise ValueError("Matrix must be square")

        # Gaussian elimination
```

```

    for i in range(m):
        max_row = max(range(i, m), key=lambda r: abs(T[r][i]))
        T[i], T[max_row] = T[max_row], T[i]

        if T[i][i] == 0:
            return float('inf')

        for j in range(n):
            T[i][j] /= T[i][i]

        for k in range(m):
            if k != i:
                factor = T[k][i]
                for j in range(n):
                    T[k][j] -= factor * T[i][j]

    return sum(1 for row in T if any(row))

def monotone_circuit_depth(f, n):
    # Placeholder function to simulate circuit depth
    # This is a dummy implementation and should be replaced with actual logic
    return random.randint(1, 5)

n = random.choice([5, 10, 15, 20, 30, 40])
f = generate_random_boolean_function(n)
T = tropicalize(f)
rho_f = rank(T)
d = monotone_circuit_depth(f, n)

lower_bound = 0.1 * d * math.log(n) # Placeholder constant c

return {
    "metric_name": "Minimal Rank",
    "metric_value": rho_f,
    "instances_tested": 1,
    "conjecture_holds": rho_f >= lower_bound,
    "counterexample": "" if rho_f >= lower_bound else f"Rank {rho_f} < {lower_bound}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 3 for i in range(5, 8)]

    results = []
    total_metric_value = 0.0
    count_supporting_conjecture = 0

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        total_metric_value += trial_result["metric_value"]
        if trial_result["conjecture_holds"]:
            count_supporting_conjecture += 1

    results.append(trial_result)

```

```

mean_metric_value = total_metric_value / len(results)
support_fraction = count_supporting_conjecture / len(results)

if support_fraction >= 0.95:
    result = f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fr
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_ho
    result = f"RESULT: FALSIFIED counterexample=\"Rank < lower_bound\" first_failing_seed={fi
else:
    result = "RESULT: INCONCLUSIVE insufficient_data"

print(result)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify if the conjecture is supported or falsified according to the pre-registered criteria. | next: Re-run the test with increased time limits and ensure it completes without crashing to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11979
2	preregistration	ollama_remote	glm4:latest	0	0	5450
3	novelty	ollama_remote	glm4:latest	0	0	4588
4	novelty	ollama_remote	glm4:latest	0	0	12769
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14661
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7896
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39706
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12082
9	judge	ollama_remote	glm4:latest	0	0	47575

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 156706 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/8beca60a4d2f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/8beca60a4d2f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/8beca60a4d2f.tar.gz (if generated)